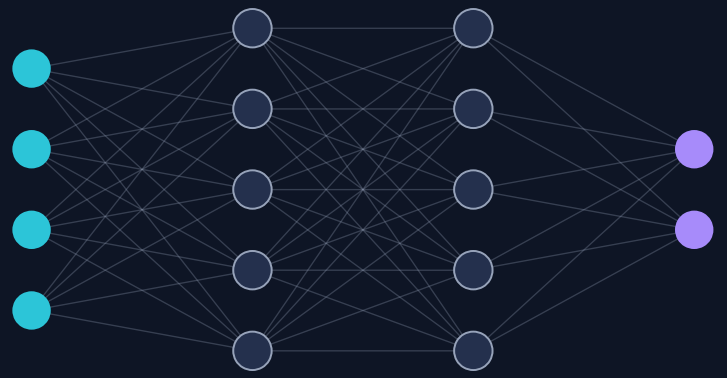
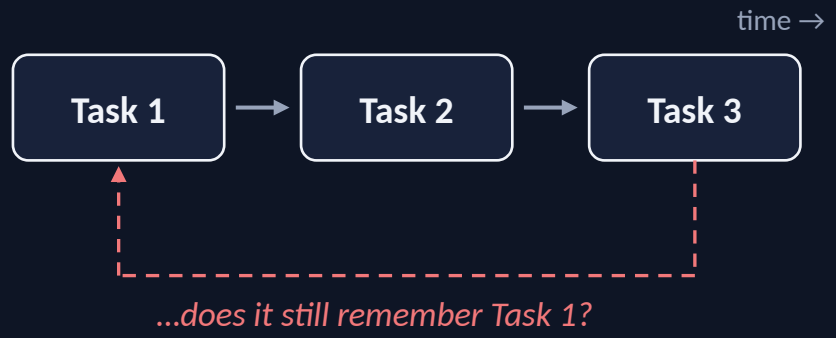
CS F317 · REINFORCEMENT LEARNING · MID-TERM

Continual Reinforcement Learning under Task Conflict



one network · shared weights · no task ID

How does an agent that learns tasks one after another forget, and how does that depend on how much the tasks clash?



Bhavya Gupta 2023A7PS0494P
Parth Kumar Garg 2023B5A71104P

Why continual learning?

A sequence of tasks

Real agents face tasks one after another, not a single fixed one

Retraining is expensive

Starting from scratch for every new task costs time and compute

Old data disappears

Data from earlier tasks is often gone by the time a new task arrives

The goal

One agent keeps learning new tasks without losing its old skills

The driving analogy



Trained: drive on the left

Retrained: drive on the right

One brain, and no sign telling the driver which country they are in. At the same junction they must pick a single habit.

Same situation, opposite correct actions.

Catastrophic forgetting

One set of weights for every task

Training follows only the current task's loss, so new learning overwrites old learning.



Schematic illustration, not measured data

Two competing pressures

Stability
Keep old skills

vs

Plasticity
Still learn new ones

A method can look stable simply because it stopped learning, so we measure both.

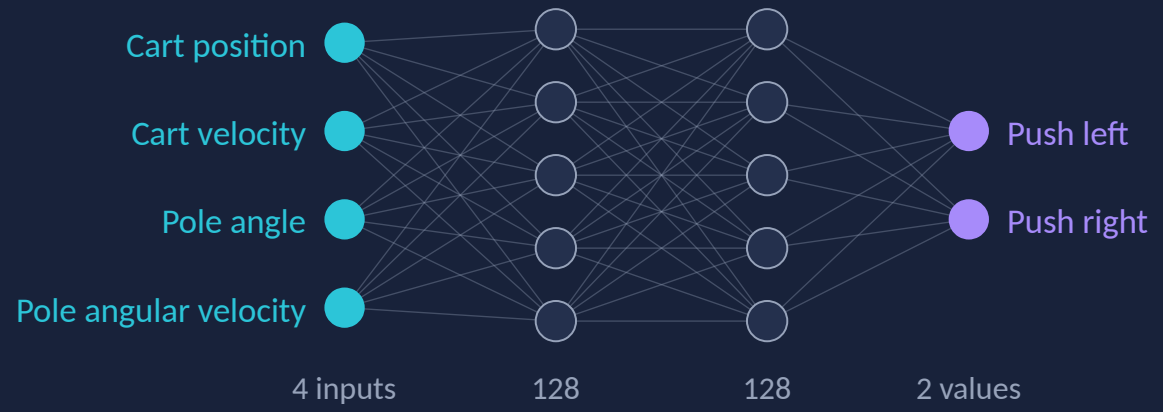
Why it is harder in RL

- The agent's own policy picks the data it sees, so the data shifts when the task shifts
- Old-task experience is gone unless it is explicitly stored

The model: DQN

Deep Q-Network

Estimates the value of each action from the current state



One network, one set of weights for all tasks. No task ID is given.

Training settings

Learning rate	**0.001**
Discount	**0.99**
Batch size	**64**
Replay buffer	**20,000**
Target network update	**every 250 steps**
Exploration ε	**1.0 → 0.05 over 5,000 steps**
Training budget	**10,000 steps per task**

Three strategies compared

BASELINE

Naive fine-tuning

Keep training on the new task. Nothing protects old skills.

No extra cost. Expected to forget.

REHEARSAL

Experience replay

Store 3,000 transitions per finished task. Every training batch is half old data, half new.

Costs memory for stored data.

REGULARIZATION

EWC

After each task, estimate how important each weight was. Then penalize changing important weights.

No stored data. Strength set by λ (λ = 100 in the main runs).

EWC for DQN is our design choice. EWC needs a probability over actions, but DQN outputs values, so we turn the values into probabilities with a softmax. This is a design choice, not a standard recipe.

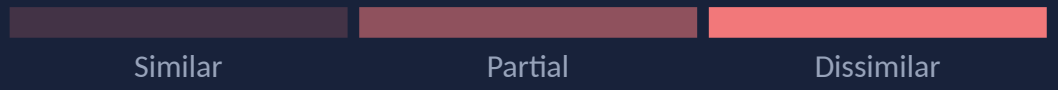
Task conflict and the question

Setting

A sequence of tasks with the same states and actions, learned one after another by one network. No going back, and no task ID.

Task conflict

How much two tasks demand different actions in the same situation.



THE QUESTION

How does forgetting change as task conflict increases, and how do replay and EWC hold up?

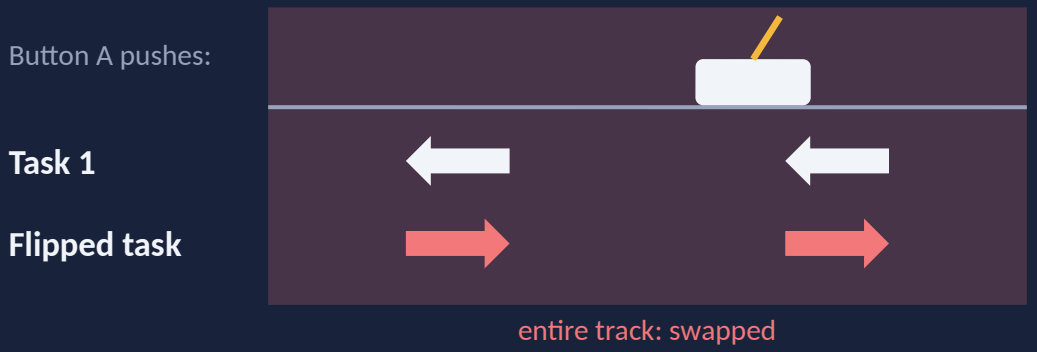
Answered with three task sequences of increasing conflict and five findings.

Full flip vs. partial flip

A policy outputs one action per situation. If Task A needs push left and Task B needs push right in the same situation, no single policy is best at both.

Full flip

Same physics as Task 1, buttons swapped everywhere



No single policy can be best at both. A stress test: failure is expected without a task ID.

Partial flip

Buttons swapped only on the right half of the track

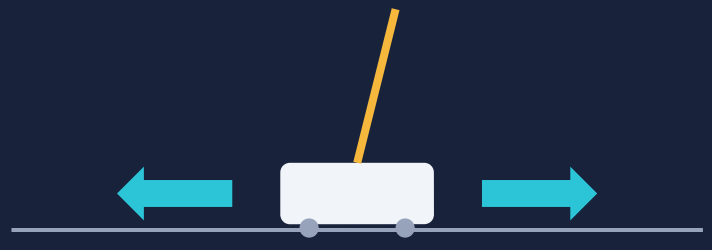


Cart position is an input, so one network can in principle learn both behaviours. The fair test.

Takeaway: full flip is a stress test where failure is expected; partial flip is the fair test of the methods.

Environment and task sequences

DONE



Push the cart left or right to keep the pole balanced.

500 max score: the number of steps survived

Tasks change the physics (gravity, pole length, push force) or swap the buttons.

Three sequences of three tasks, ordered by conflict. All start from the same default game.

		T1	T2	T3
Similar	● ○ ○	Default game	Small physics change gravity 9.8 → 12 pole 0.5 → 0.9	Small physics change
Partial	● ● ○	Default game	Big physics change gravity 15, pole 1.0, force 8	Buttons swapped on right half
Dissimilar	● ● ●	Default game	Long pole, weak push pole 1.5, force 6	Buttons swapped everywhere

Why three? One sequence gives one data point; three conflict levels show how forgetting changes as conflict grows.

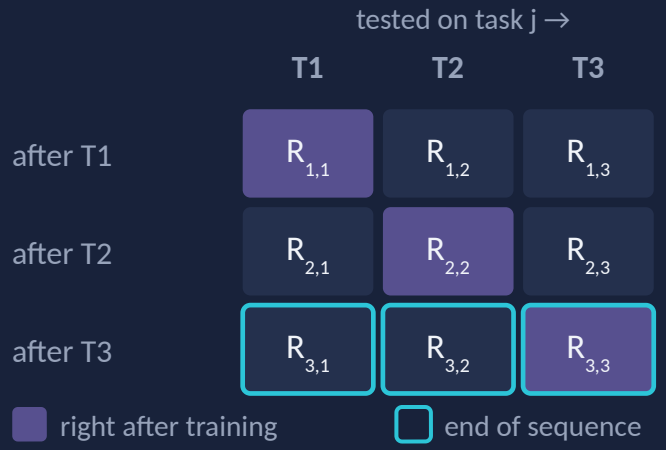
Training protocol and metrics

DONE

Fresh untrained agent → Train on T1 → Test on all tasks → Train on T2 → Test on all tasks → Train on T3 → Test on all tasks

3 sequences × 3 strategies × 3 seeds = 27 main runs, plus an EWC strength sweep · 10,000 steps per task · 20 evaluation episodes

The score table



Forgetting

Score right after learning a task minus score at the end

Plasticity

Score on each task right after training on it

Catches methods that stop learning

Forward transfer

Head start on a task from earlier learning

Average final return

Mean score over all tasks at the end of the sequence

What has been done

DONE

27

main runs complete

3 sequences × 3 strategies × 3 seeds, plus an EWC strength sweep

Identical settings for every main run

3 seeds
10,000 steps per task
20 evaluation episodes
EWC λ 100

End-to-end pipeline

Task variants, training loop, evaluation, metrics, plotting

Three strategies

Naive, replay (3,000 transitions per task), EWC

Three task sequences

Similar, partial and dissimilar, ordered by conflict

Plasticity metric

So a method cannot look good by never learning

EWC strength sweep

λ 1,000 and 10,000 on the dissimilar sequence

EWC diagnostics

Importance by layer and a softmax saturation check

Similar sequence at the full training budget: finished

All three sequences now use identical settings, so their numbers compare directly.

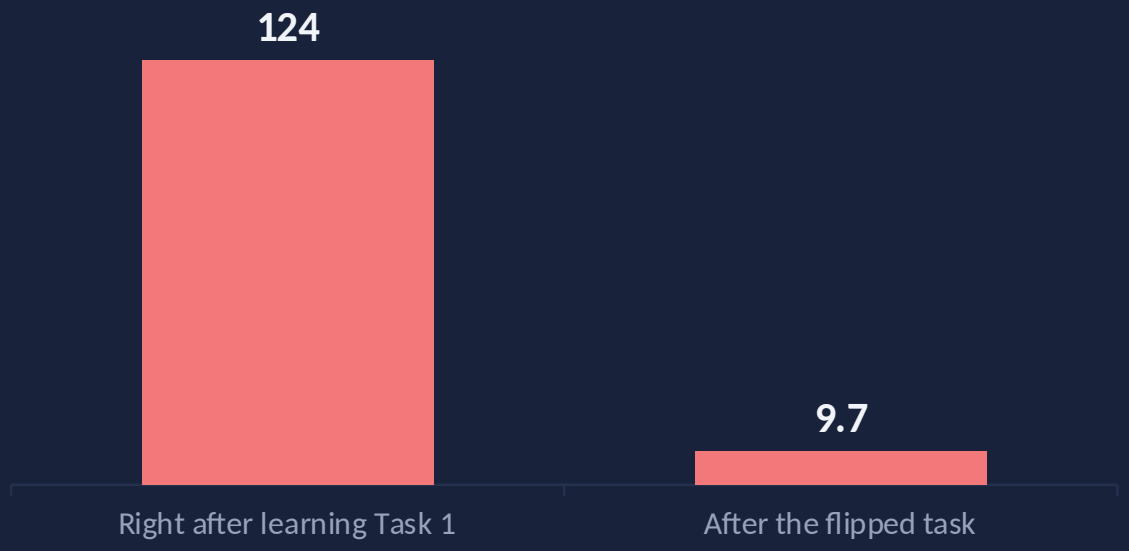
DONE

Full conflict wipes out Task 1

RESULTS · 3 SEEDS

Task 1 score, naive fine-tuning

Mean of 3 seeds · maximum possible 500



Per seed: 125, 110, 135 after learning it; 9.4, 10.2, 9.4 at the end

≈ 9

What a policy that always pushes one way scores. The old skill is gone.

19.5

The long-pole task (Task 2) is lost too: it fell to 19.5 in all three seeds.

≈ 115

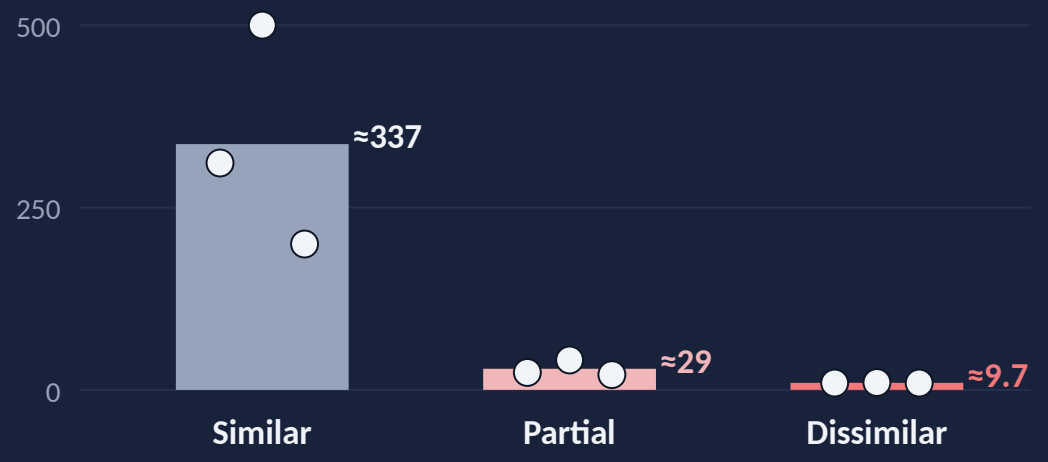
The flipped task (Task 3) is learned only partly: about 115 out of 500.

Forgetting grows with conflict

RESULTS · 3 SEEDS

Task 1 score at the end of the sequence

Bar = mean of 3 seeds · dot = one seed



Task 1 score as training proceeds

Mean of 3 seeds · starts at ≈124 in every sequence



Forgetting even without conflict. In one similar-sequence seed, Task 2 fell from 500 to 100 after the third task.

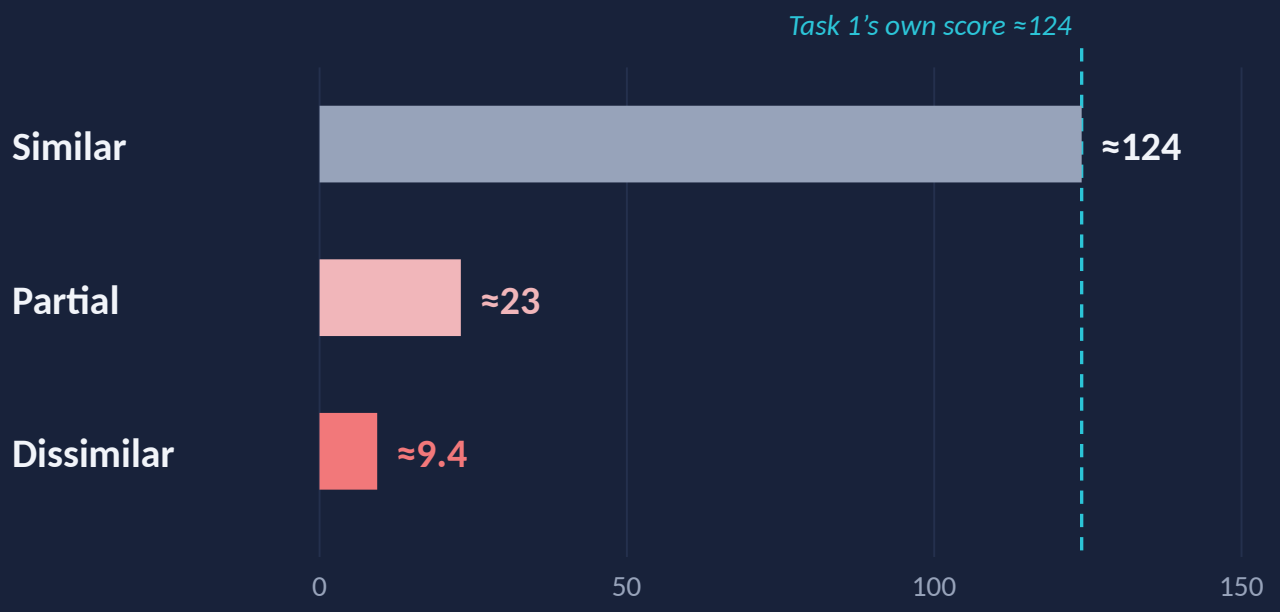
Why Task 1's final score? The forgetting metric depends on how high a task peaked, so it doesn't track conflict cleanly (naive: −53, 207, 160).

Conflict between tasks is graded

RESULTS · 3 SEEDS

How well does a Task 1 policy do on the last task?

Policy trained only on Task 1, tested on the last task before any training on it · mean of 3 seeds



≈9 is what always pushing one way scores

What it shows

The more the tasks conflict, the less a Task 1 policy can do on the new task.

On the similar sequence it scores about the same as on Task 1 itself: skills carry over.

This confirms the three sequences are three different conflict levels.

Replay and EWC don’t reliably help

RESULTS · 3 SEEDS

Task 1 score at the end · mean of 3 seeds

Sequence	Naive	Replay	EWC (λ 100)
Similar	337	348	136
Partial	29	182 * seeds 25 / 497 / 25	37
Dissimilar	9.7	38 seeds 88 / 16 / 10	11

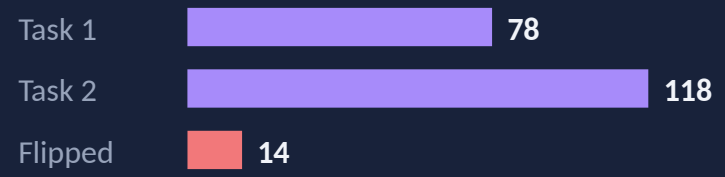
* Replay’s partial mean comes from a single seed (497); the other two ended at 25.

Stronger EWC, dissimilar

Task 1 at the end · sweep used 10 evaluation episodes (main table: 20)

λ 1,000	≈ 12
λ 10,000	≈ 38

One λ 10,000 seed, final scores



Kept old tasks, barely learned the new one: the stability–plasticity trade-off.

⚖ **No winner claimed.** Seed-to-seed variation is as large as the gaps between methods.

Why EWC struggles

RESULTS · 3 SEEDS

A cost of protection

Similar sequence · average final return · mean of 3 seeds



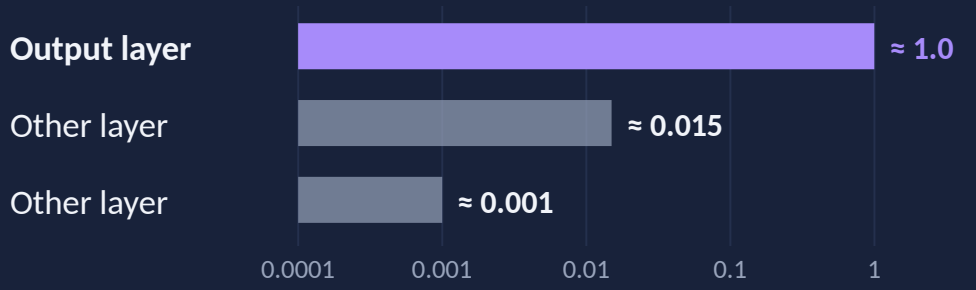
EWC ended lower than naive in all three seeds.

HYPOTHESIS

The penalty may slow learning when there is little to protect. From 3 seeds only.

EWC importance by layer

Average importance after Task 1 · one run · log scale



The output layer decides left versus right, which is exactly what the flipped task must rewrite.

Softmax not saturated (mean top probability 0.64), so the importance estimate itself works.

Hypothesis, not proven: EWC protects the weights the flipped task has to change.

Summary and limitations

1

Forgetting is reproduced, and it grows with conflict

Naive Task 1 final score: ≈337 similar → ≈29 partial → ≈9.7 dissimilar

2

Conflict between the sequences is graded

Task 1 policy on the last task, before training: ≈124 → ≈23 → ≈9.4

3

Replay and EWC don’t reliably fix it

At the tested settings, seed-to-seed variation is as large as the gaps between methods

Limitations

- Toy environments: small CartPole variants
- Only 3 seeds per setting
- DQN only
- EWC importance for DQN is a design choice
- EWC strength sweep only on the dissimilar sequence

Next steps

PLANNED

1

Measure conflict directly

Measure conflict inside the network rather than by label, and relate it to forgetting.

2

Baselines & ablations

Add stronger comparison methods and vary the key settings.

3

Conflict-guided replay

Test whether using measured conflict helps replay forget less.

REFERENCES

Mnih et al. Human-level control through deep reinforcement learning. Nature, 2015.

Kirkpatrick et al. Overcoming catastrophic forgetting in neural networks. PNAS, 2017.

Chaudhry, Ranzato, Rohrbach & Elhoseiny. Efficient lifelong learning with A-GEM. ICLR, 2019.

Chung, Cherif, Meger & Precup. Parseval regularization for continual reinforcement learning. NeurIPS, 2024.

The goal: understand how task conflict drives forgetting in one network with no task ID, and whether measuring conflict lets an agent forget less.